Python Dictionary Methods

codecademy

Dictionary Key-Value Methods

When trying to look at the information in a Python dictionary, there are multiple methods that return objects that contain the dictionary keys and values.

- `.keys()` returns the keys through a `dict_keys` object.
- `.values()` returns the values through a `dict_values` object.
- `.items()` returns both the keys and values through a `dict_items` object.

```
ex_dict = {"a": "anteater", "b": "bumblebee", "c": "cheetah"}

ex_dict.keys()
# dict_keys(["a","b","c"])

ex_dict.values()
# dict_values(["anteater", "bumblebee", "cheetah"])

ex_dict.items()
# dict_items([("a","anteater"),("b","bumblebee"),("c","cheetah")])
```

get() Method for Dictionary

Python provides a `.get()` method to access a `dictionary` value if it exists. This method takes the `key` as the first argument and an optional default value as the second argument, and it returns the value for the specified `key` if `key` is in the dictionary. If the second argument is not specified and `key` is not found then `None` is returned.

```
# without default
{"name": "Victor"}.get("name")
# returns "Victor"

{"name": "Victor"}.get("nickname")
# returns None

# with default
{"name": "Victor"}.get("nickname", "nickname is not a key")
# returns "nickname is not a key"
```

Merging dictionaries with the `.update()` method in Python

Given two dictionaries that need to be combined, Python makes this easy with the $.\text{update}()$ function. For $\text{dict1.update(dict2)}$, the key-value pairs of dict2 will be written into the dict1 dictionary. For keys in *both* dict1 and dict2, the value in dict1 will be overwritten by the corresponding value in dict2.

```
dict1 = {'color': 'blue', 'shape':
'circle'}
dict2 = {'color': 'red', 'number': 42}

dict1.update(dict2)

# dict1 is now {'color': 'red', 'shape':
'circle', 'number': 42}
```

The `.pop()` Method for Dictionaries in Python

Python dictionaries can remove key-value pairs with the $.\text{pop}()$ method. The method takes a key as an argument and removes it from the dictionary. At the same time, it also returns the value that it removes from the dictionary.

```
famous_museums = {'Washington':
'Smithsonian Institution', 'Paris': 'Le
Louvre', 'Athens': 'The Acropolis
Museum'}
famous_museums.pop('Athens')
print(famous_museums) # {'Washington':
'Smithsonian Institution', 'Paris': 'Le
Louvre'}
```

Print